

Informe Previo TFG: Agentes Autónomos y Sistemas Multi-Agente (MAS)



Samuel Gómez Grande

6 de abril de 2026

Índice

1. Introducción	2
2. Fundamentos: De LLMs Individuales a Sistemas Multi-Agente (MAS)	2
2.1. ¿Qué es un Modelo de Lenguaje Extenso (LLM)?	2
2.2. El motor tecnológico: La arquitectura Transformer	3
2.3. ¿Qué es un Agente Inteligente?	3
2.4. ¿Qué es un Sistema Multi-Agente (MAS)?	4
2.5. Limitaciones del agente individual	4
2.6. El paradigma de los Sistemas Multi-Agente (MAS)	5
3. Arquitectura y Flujo de Trabajo de los MAS	5
3.1. Interfaz con el Entorno (Percepción)	6
3.2. Perfilado de Agentes (Profiling)	6
3.3. Razonamiento y Acción Autónoma (Self-action)	7
3.4. Interacción Mutua y Comunicación	7
3.5. Evolución y Aprendizaje	8
4. Conectividad con el Mundo Real: Model Context Protocol (MCP)	8
4.1. Arquitectura general de MCP	9
4.2. Capacidades expuestas: recursos, herramientas y prompts	9
4.3. Flujo operativo en un sistema multiagente	10
4.4. Seguridad, permisos y gobernanza	10
4.5. Retos actuales y buenas prácticas	10
5. Aplicaciones Prácticas de los MAS basados en LLM	10
5.1. Resolución de Problemas	10
5.2. Frameworks para llevar los MAS a escenarios reales	11
6. Propuesta realizada con el framework CrewAI	12
6.1. Instalación del entorno	12
6.2. Creación y ejecución de un proyecto base	12
6.3. Configuración de la prueba: dos agentes y dos tareas	15
6.4. Resultado observado en la ejecución	16
7. Conclusión	16

1. Introducción

En las últimas décadas, la Inteligencia Artificial (IA) ha pasado de ser un campo teórico para convertirse en un pilar fundamental de la actividad humana. Históricamente, los sistemas inteligentes estaban dominados por enfoques basados en aprendizaje por refuerzo o sistemas expertos con reglas predefinidas. En estos entornos, los agentes ejecutaban acciones simples y bien delimitadas; sin embargo, ante situaciones no previstas, presentaban limitaciones críticas en adaptabilidad y manejo de la complejidad, lo que impedía gestionar la incertidumbre del mundo real.

El panorama ha cambiado drásticamente con la irrupción de los *Large Language Models* (LLMs). Esta tecnología no se limita a la generación de texto, sino que ha impulsado la creación de agentes autónomos capaces de percibir su entorno, razonar y ejecutar acciones de forma independiente. Para comprender este impacto, es fundamental analizar la evolución hacia los Sistemas Multi-Agente (MAS).

A diferencia de los MAS tradicionales, dependientes de protocolos de comunicación rígidos, los sistemas actuales basados en LLM aprovechan el lenguaje natural para colaborar. Esto permite que múltiples agentes con identidades y roles distintos funcionen como un equipo cohesionado, capaz de debatir y resolver problemas imprevistos.

2. Fundamentos: De LLMs Individuales a Sistemas Multi-Agente (MAS)

Al igual que sucede en la vida real, el trabajo en equipo es mucho más eficiente, rápido y astuto que una única persona trabajando en solitario. Mientras que un individuo puede verse superado por una tarea compleja o cometer errores sin que nadie lo corrija, la coordinación entre agentes permite repartir el trabajo y resolver problemas de forma más inteligente.

2.1. ¿Qué es un Modelo de Lenguaje Extenso (LLM)?

Un LLM (*Large Language Model*) es un sistema de Inteligencia Artificial diseñado para entender, generar y manipular el lenguaje humano a gran escala. A diferencia de los programas de procesamiento de texto tradicionales, un LLM no sigue reglas gramaticales rígidas, sino que aprende patrones complejos a partir de una cantidad masiva de datos.

2.2. El motor tecnológico: La arquitectura Transformer

Para que un LLM sea capaz de razonar y colaborar en un sistema multi-agente, necesita una estructura interna que le permita entender el contexto de forma global. Esta estructura es la arquitectura **Transformer**, introducida en 2017, que supuso una ruptura con los modelos clásicos que procesaban la información de manera secuencial (palabra por palabra).

El corazón del Transformer es el mecanismo de **auto-atención (self-attention)**. Simplificándolo, este mecanismo permite que el modelo, a la hora de leer un texto, pueda leer todas las palabras del texto simultáneamente para entender las relaciones de importancia entre ellas. Por ejemplo, en un entorno de agentes, si un agente recibe la instrucción “*Analiza el código y corrígelo*”, el Transformer permite que el modelo identifique que “*corrígelo*” se refiere específicamente al “*código*” mencionado antes, manteniendo la coherencia incluso en textos muy largos.

Esta capacidad de gestionar relaciones complejas y contextos extensos es, en última instancia, lo que permite que los agentes no solo generen texto, sino que mantengan un hilo lógico de pensamiento y planificación durante la resolución de problemas en equipo.

2.3. ¿Qué es un Agente Inteligente?

Un **Agente Inteligente** es una entidad capaz de percibir su entorno, procesar esa información y ejecutar acciones de manera autónoma para alcanzar un objetivo específico.

Históricamente, los agentes inteligentes funcionaban bajo el paradigma —*Si ocurre A, entonces haz B*—; sin embargo, el enfoque moderno basado en LLM ha transformado este concepto en un sistema de razonamiento dinámico. En este nuevo paradigma, la **Arquitectura Transformer** actúa como el núcleo cognitivo del agente. Gracias al mecanismo de *self-attention*, el agente no solo “lee” palabras, sino que comprende la relación jerárquica y contextual entre ellas, permitiéndole ir más allá de las reglas preprogramadas mediante tres capacidades fundamentales:

- **Planificación y Razonamiento (Planning):** Utilizando su arquitectura Transformer para procesar secuencias de pensamiento, el agente descompone objetivos complejos en pasos lógicos intermedios. Esto le permite anticipar necesidades y corregir su estrategia sobre la marcha.
- **Uso de Herramientas (Tool Use):** El agente identifica, mediante la comprensión del lenguaje, cuándo sus capacidades internas son insuficientes y decide de forma autónoma invocar herramientas externas (APIs, intérpretes de código o buscadores) para devolver una respuesta coherente y verídica.

- **Gestión de Memoria:** Los agentes actuales emulan la cognición humana gestionando una *memoria a corto plazo* (la ventana de contexto del Transformer) y una *memoria a largo plazo* (mediante bases de datos vectoriales) para recuperar conocimientos específicos.

Esta evolución convierte al agente en un sistema **proactivo** en lugar de reactivo, dotándolo de una “intuición” digital para navegar por la ambigüedad del lenguaje y la complejidad de las tareas técnicas.

2.4. ¿Qué es un Sistema Multi-Agente (MAS)?

Un **Sistema Multi-Agente (MAS)** es un entorno compuesto por múltiples agentes inteligentes que interactúan entre sí para alcanzar objetivos que superan las capacidades de una entidad individual. Si un agente basado en Transformer es una unidad de razonamiento aislada, el MAS constituye una **estructura colaborativa** donde la inteligencia y la ejecución de tareas se distribuyen entre diferentes nodos.

El contraste fundamental respecto a los sistemas de software tradicionales radica en su flexibilidad operativa. Los sistemas actuales aprovechan el lenguaje natural para que los agentes puedan coordinarse, debatir soluciones y compartir información de forma dinámica.

En un MAS basado en LLM, la clave no es solo la potencia de procesamiento de cada modelo, sino la **arquitectura de comunicación** que los une. Esto permite transformar una simple herramienta de consulta en una fuerza de trabajo organizada donde los agentes no solo ejecutan órdenes, sino que colaboran activamente para resolver problemas complejos mediante la especialización y la supervisión mutua.

2.5. Limitaciones del agente individual

Aunque un único agente puede ser eficiente para tareas sencillas, presenta limitaciones críticas en entornos complejos:

- **Capacidad restringida:** Límite físico en su ventana de contexto y potencia de cálculo.
- **Alucinaciones y falta de revisión:** Riesgo de generar información errónea sin contraste externo.
- **Dificultad para la autocorrección:** Tendencia a incurrir en sesgos cognitivos sin una “segunda opinión”.
- **Falta de especialización:** Un perfil generalista es menos eficiente que un equipo

de expertos dedicados.

- **Dificultad en la planificación:** Tendencia a perder la coherencia en procesos de largo plazo.

2.6. El paradigma de los Sistemas Multi-Agente (MAS)

El paradigma MAS traslada el enfoque desde la potencia de un único modelo hacia la colaboración. Los pilares que definen este esquema son:

- **Especialización de roles (Role Playing):** Asignación de personalidades y capacidades específicas (ej. “crítico”, “programador”).
- **Razonamiento iterativo y debate:** Diálogo entre agentes para detectar errores y elevar la calidad.
- **División de tareas (Task Decomposition):** Fragmentación de objetivos ambiciosos en pasos manejables.
- **Memoria compartida y comunicación:** Alineación con el objetivo global mediante intercambio de información.
- **Escalabilidad y robustez:** Resistencia a fallos mediante la redistribución de tareas.

3. Arquitectura y Flujo de Trabajo de los MAS

Para que un Sistema Multi-Agente (MAS) sepa resolver problemas complejos de manera autónoma, necesita una estructura de organización clara. La literatura científica actual propone un flujo de trabajo unificado compuesto por módulos fundamentales que guían el ciclo de vida de los agentes.

A continuación, se detallan los componentes clave de esta arquitectura: la extracción de información del entorno (**Percepción**), el diseño de roles específicos (**Perfilado**), los procesos cognitivos para la toma de decisiones (**Acción Autónoma**) y las estructuras de comunicación para colaborar (**Interacción Mutua**).

La estructura de esta sección (percepción, *profiling*, *self-action*, interacción mutua y evolución) está alineada con la literatura reciente sobre MAS basados en LLM, donde estos cinco bloques se plantean como núcleo operativo del sistema. Asimismo, la visión de agentes autónomos a nivel individual encaja con los marcos actuales que combinan planificación, ejecución, evaluación y coordinación.

3.1. Interfaz con el Entorno (Percepción)

En un sistema multiagente, la percepción es la capa que conecta a los agentes con el estado real del problema. No se trata solo de recibir datos”, sino de transformar señales heterogéneas en contexto operativo: entradas de usuario, resultados de herramientas externas, documentos, eventos del sistema o respuestas de otros agentes. Esta interfaz actúa como un filtro inteligente que decide qué información es relevante en cada momento y en qué formato debe circular dentro del sistema.

Desde un punto de vista arquitectónico, una buena interfaz de percepción cumple tres funciones clave:

1. Normaliza entradas para que agentes distintos trabajen sobre una representación común y no sobre fragmentos inconsistentes.
2. Reduce ruido: prioriza señales útiles, elimina redundancias y evita sobrecargar la toma de decisiones con datos irrelevantes.
3. Mantiene trazabilidad, de modo que cada acción posterior pueda justificarse con evidencia observable y no solo con inferencias internas.

Esta capa suele materializarse como un conjunto de conectores y adaptadores: APIs, bases de conocimiento, buscadores, sistemas de archivos o servicios corporativos. Cuanto mejor diseñada esté esta frontera con el entorno, más robusto será el comportamiento del MAS ante cambios, ambigüedad o información incompleta. Por eso, la percepción no es un bloque auxiliar, sino el punto donde empieza la calidad global del sistema: si la lectura del contexto falla, la planificación y la ejecución también lo harán.

3.2. Perfilado de Agentes (Profiling)

Una vez que el sistema puede leer el entorno con suficiente calidad, el siguiente paso es definir qué tipo de agente participa y qué se espera de él. El **profiling** consiste precisamente en eso: establecer identidad funcional, nivel de autonomía, límites de actuación y criterios de calidad. En lugar de crear agentes ”genéricos” que intentan hacerlo todo, se configuran perfiles con responsabilidades claras, lo que reduce ambigüedad y mejora la coordinación global.

Un perfil bien diseñado suele incluir cuatro elementos: rol principal (por ejemplo, analista, planificador o revisor), alcance de decisiones (qué puede resolver por sí mismo y qué debe escalar), herramientas disponibles (fuentes de datos, APIs, memoria compartida) y estilo de salida esperado (más sintético, más argumentado, orientado a ejecución, etc.). Esta definición previa evita solapamientos entre agentes, disminuye respuestas contradictorias y facilita la evaluación del rendimiento de cada componente del sistema.

Además, el **profiling** no es estático. En escenarios reales, los perfiles se ajustan según métricas operativas: tiempo de respuesta, tasa de errores, calidad de las propuestas o nivel de consenso alcanzado en tareas colaborativas. Esto permite iterar la arquitectura sin rediseñar todo el MAS desde cero. En términos prácticos, perfilar bien no solo organiza mejor el trabajo interno, sino que convierte al sistema en una estructura más predecible, mantenible y escalable.

3.3. Razonamiento y Acción Autónoma (Self-action)

El bloque de *self-action* es donde cada agente deja de ser un "nodo pasivo" y se convierte en una unidad de trabajo real. Aquí no solo interpreta una instrucción, sino que encadena decisiones: analiza el contexto disponible, propone un plan de actuación, ejecuta pasos concretos y revisa si el resultado cumple el objetivo marcado. Esta secuencia hace que el comportamiento del agente sea más cercano a un ciclo operativo que a una única respuesta aislada.

En términos funcionales, esta autonomía suele apoyarse en un bucle corto de trabajo: observar, decidir, actuar y verificar. Si la verificación detecta desviaciones, el agente re-planifica sin necesidad de reiniciar todo el proceso. Este mecanismo reduce bloqueos en tareas largas y permite mantener continuidad incluso cuando aparece información nueva o contradictoria durante la ejecución. Este tipo de ciclo también se describe en los surveys recientes sobre agentes autónomos basados en LLM, donde se enfatiza la necesidad de combinar planificación, ejecución y evaluación en una misma unidad operativa.

También es importante delimitar el grado de autonomía. Un agente puede tener permiso para ejecutar acciones de bajo riesgo de forma directa (por ejemplo, clasificar información o generar borradores), pero requerir validación externa en decisiones críticas. Ese equilibrio entre iniciativa y control evita dos extremos problemáticos: agentes excesivamente dependientes que frenan el sistema, o agentes sobreactivos que actúan sin suficiente supervisión. En un MAS bien diseñado, el *self-action* no busca "hacer más", sino actuar con criterio, trazabilidad y coherencia con el objetivo global.

3.4. Interacción Mutua y Comunicación

La interacción entre agentes es el mecanismo que convierte capacidades individuales en inteligencia colectiva. Cuando esta comunicación está bien diseñada, el sistema no solo comparte información: construye criterio común, detecta inconsistencias y coordina prioridades sin depender de un único punto de control. En otras palabras, los agentes no trabajan ".^{en} paralelo" de forma aislada, sino en cooperación estructurada.

Para que esto funcione, la comunicación debe ser útil y disciplinada. Cada mensaje debería

aportar estado de tarea, justificación breve y siguiente paso propuesto. Si los intercambios son vagos o excesivos, aparece el conocido problema de sobre coste conversacional: se consume tiempo en hablar, pero no en resolver. Por eso muchos MAS incorporan reglas explícitas de comunicación, como turnos de intervención, formatos de salida o protocolos de escalado cuando hay desacuerdo técnico.

Un aspecto especialmente relevante es la gestión del conflicto entre agentes. El desacuerdo no siempre es un fallo; bien gestionado, es una fuente de robustez. Un agente puede cuestionar supuestos, otro aportar evidencia y un tercero consolidar una decisión final. Este patrón de contraste mejora la calidad de la solución y reduce errores silenciosos. En consecuencia, la comunicación interna no debe evaluarse solo por su volumen, sino por su capacidad de alinear decisiones, acelerar la convergencia y sostener coherencia operativa en tareas complejas.

3.5. Evolución y Aprendizaje

El último componente del flujo es la capacidad del sistema para mejorar con la experiencia. En un MAS, evolucionar no significa únicamente “acumular historial”, sino transformar resultados pasados en ajustes concretos: redefinir roles, afinar reglas de coordinación, actualizar criterios de validación y corregir patrones que hayan generado errores repetidos.

Este aprendizaje puede operar en dos escalas. A corto plazo, los agentes refinan su comportamiento durante una misma sesión, por ejemplo adaptando estrategias cuando detectan que una ruta de resolución no converge. A medio plazo, el sistema incorpora aprendizajes persistentes a través de memoria estructurada, métricas de desempeño y revisiones periódicas de arquitectura. Esta doble escala permite combinar adaptabilidad inmediata con mejora sostenida.

Para que la evolución sea útil, es clave medir con indicadores claros: calidad de respuesta, tiempo de resolución, tasa de retrabajo, número de conflictos no resueltos o estabilidad de las decisiones finales. Sin evaluación, no hay aprendizaje verificable; solo cambios difíciles de justificar. Por eso, en entornos reales, un MAS maduro se comporta como un sistema vivo pero gobernado: se adapta, sí, pero lo hace con control, evidencia y trazabilidad.

4. Conectividad con el Mundo Real: Model Context Protocol (MCP)

La conexión entre agentes y sistemas externos se ha estandarizado recientemente mediante el *Model Context Protocol* (MCP), un protocolo abierto que permite a aplicaciones de IA integrarse con fuentes de datos, herramientas y flujos de trabajo de forma uniforme. En la

práctica, MCP actúa como una interfaz común entre clientes de IA y servicios externos, reduciendo integraciones ad hoc y mejorando portabilidad entre ecosistemas. Su valor principal es desacoplar la lógica del agente de los detalles de conexión, lo que facilita construir una vez y reutilizar capacidades en distintos entornos (por ejemplo, asistentes, IDEs o plataformas empresariales).

En este contexto, MCP es especialmente relevante porque proporciona una base técnica realista para que un MAS salga del entorno puramente conversacional y pueda consultar información, ejecutar acciones y mantener trazabilidad sobre recursos externos bajo un esquema estándar.

4.1. Arquitectura general de MCP

MCP sigue una arquitectura cliente-servidor. El cliente suele ser la aplicación de IA (o el agente que ejecuta la tarea), mientras que el servidor MCP actúa como puente hacia servicios externos. Esta separación simplifica el diseño porque el agente no necesita implementar una integración distinta para cada herramienta: delega esa responsabilidad en servidores especializados.

Desde un punto de vista de ingeniería, esta arquitectura también mejora mantenibilidad. Si cambia una API externa, el ajuste se concentra en el servidor MCP correspondiente, sin obligar a rediseñar toda la lógica del sistema multiagente. El resultado es una capa de integración más estable y menos costosa de evolucionar.

4.2. Capacidades expuestas: recursos, herramientas y prompts

Una ventaja práctica de MCP es que organiza la conexión externa en capacidades bien definidas. De forma simplificada, se pueden distinguir tres tipos de piezas:

1. **Recursos:** información consultable (documentos, registros, datos estructurados).
2. **Herramientas:** acciones ejecutables (consultas, transformaciones, automatizaciones).
3. **Prompts:** plantillas o flujos guiados que estandarizan cómo se formula una tarea.

Esta separación ayuda a que los agentes no mezclen acceso a datos con ejecución de acciones. Además, facilita auditar qué se leyó, qué se ejecutó y bajo qué contexto se tomó cada decisión.

4.3. Flujo operativo en un sistema multiagente

Cuando un agente necesita contexto externo, el flujo habitual es: identificar la necesidad de información o acción, seleccionar la capacidad MCP adecuada, ejecutar la llamada y reincorporar el resultado al razonamiento del equipo. Aunque parezca simple, este ciclo introduce una mejora clave: cada paso deja rastro y puede validarse.

En sistemas con varios agentes, MCP permite que distintas especializaciones trabajen sobre una base común de conectividad. Por ejemplo, un agente de análisis puede consultar datos y otro agente de control puede verificar la consistencia antes de aprobar una acción. Este patrón reduce decisiones opacas y mejora la coordinación técnica.

4.4. Seguridad, permisos y gobernanza

La conexión a sistemas reales implica riesgos, por lo que la gobernanza es un componente central. No todos los agentes deben tener el mismo nivel de acceso: lo recomendable es aplicar principio de mínimo privilegio, separar permisos de lectura y escritura, y registrar cada operación sensible.

También conviene introducir validaciones antes de ejecutar acciones críticas: límites de alcance, comprobaciones de formato, confirmaciones humanas en operaciones de alto impacto y políticas de uso por tipo de herramienta. En la práctica, un MAS conectado por MCP es tan fiable como lo sea su política de control de acceso y trazabilidad.

4.5. Retos actuales y buenas prácticas

Aunque MCP simplifica integraciones, no elimina todos los problemas. Persisten retos como la calidad desigual de fuentes externas, la latencia en cadenas de herramientas y la necesidad de manejar errores de forma robusta cuando un servicio no responde.

Entre las buenas prácticas más útiles destacan: diseñar contratos de entrada/salida claros, aplicar reintentos controlados, desacoplar tareas largas en pasos pequeños y monitorizar métricas operativas (tiempo de respuesta, tasa de fallo, frecuencia de replanificación). Con estas medidas, MCP deja de ser solo un conector y pasa a convertirse en una base sólida para operaciones multiagente en escenarios reales.

5. Aplicaciones Prácticas de los MAS basados en LLM

5.1. Resolución de Problemas

Más allá de su interés teórico, los Sistemas Multi-Agente basados en LLM empiezan a ser relevantes cuando se trasladan a escenarios concretos. Los MAS, tal y como se entienden

en esta época, muestran que su valor no está solo en generar texto, sino en coordinar procesos, repartir responsabilidades y sostener una resolución de tareas más flexible que la de un único modelo aislado.

En este contexto, las aplicaciones más interesantes no son las más espectaculares, sino las que aprovechan bien la división del trabajo. Un MAS permite que un agente analice, otro contraste, otro reescriba o valide, y que todo ello ocurra dentro de un flujo más o menos ordenado. Esto resulta especialmente útil cuando la tarea no se resuelve con una única respuesta, sino con varias decisiones encadenadas. Es por esta razón por la que estos sistemas encajan bien en entornos donde hay incertidumbre, múltiples pasos intermedios o necesidad de revisar resultados antes de dar una salida final.

5.2. Frameworks para llevar los MAS a escenarios reales

Para que un Sistema Multi-Agente deje de ser una idea teórica y pueda evaluarse en condiciones cercanas al uso real, es habitual apoyarse en frameworks especializados. Estas herramientas reducen la complejidad inicial de implementación y permiten centrarse en lo verdaderamente importante: definir roles, coordinar tareas, controlar la calidad de las respuestas y medir resultados.

Permiten modelar flujos de trabajo con varios agentes, integrar fuentes de datos externas y establecer reglas de interacción entre componentes. De este modo, es posible pasar de una descripción conceptual del MAS a una prueba funcional en la que se observan tiempos de respuesta, consistencia de decisiones y capacidad de coordinación.

Entre los frameworks más utilizados destacan los siguientes:

- **LangGraph**: orientado a construir flujos de agentes con estados y transiciones explícitas, especialmente útil cuando se necesitan procesos largos, controlados y trazables.
- **LlamaIndex**: centrado en integrar conocimiento externo con LLM, facilitando la ingesta, organización y consulta de documentos y bases de datos.
- **smolagents**: propuesta ligera para prototipado rápido, con menor complejidad inicial y buena adaptación a pruebas de concepto.
- **AutoGen**: framework popular para conversaciones multiagente con roles diferenciados, útil para coordinación entre agentes que colaboran y se supervisan entre sí.
- **CrewAI**: centrado en la colaboración por roles y tareas dentro de una *crew*, con una definición clara de responsabilidades y secuencias de trabajo.

6. Propuesta realizada con el framework CrewAI

Como primer paso de la propuesta, se preparó el entorno de trabajo para poder ejecutar una prueba funcional con CrewAI. La instalación se realizó siguiendo la documentación oficial, priorizando una configuración simple y reproducible.

6.1. Instalación del entorno

Antes de instalar CrewAI, se verificó que la versión de Python fuese compatible (≥ 3.10 y < 3.14):

```
python --version
```

Después, se instaló uv, que es el gestor recomendado por CrewAI. En Windows, el comando indicado en la guía es:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Con uv disponible, se instaló la CLI de CrewAI:

```
uv tool install crewai
```

Si aparece un aviso de PATH, se puede ejecutar:

```
uv tool update-shell
```

Para confirmar que la instalación quedó correcta:

```
uv tool list
```

6.2. Creación y ejecución de un proyecto base

Una vez instalada la herramienta, se puede crear la estructura inicial del proyecto con:

```
crewai create crew tfg
```

```
PS C:\Users\samue\Desktop\hola> crewai create crew tfg
Creating folder tfg...
Select a provider to set up:
1. openai
2. anthropic
3. gemini
4. nvidia_nim
5. groq
6. huggingface
7. ollama
8. watson
9. bedrock
10. azure
11. cerebras
12. sambanova
13. other
q. Quit
Enter the number of your choice or 'q' to quit: 5
```

Figura 1: Selección del proveedor antes de la elección del modelo.

En esta fase se optó por **Groq** por ser una opción gratuita y por ofrecer, en las pruebas iniciales realizadas, un comportamiento más estable que otras alternativas como Gemini. En cualquier caso, la elección final del modelo para el desarrollo completo del TFG queda abierta para una fase posterior.

```
Enter the number of your choice or 'q' to quit: 5
Select a model to use for Groq:
1. groq/llama-3.1-8b-instant
2. groq/llama-3.1-70b-versatile
3. groq/llama-3.1-405b-reasoning
4. groq/gemma2-9b-it
5. groq/gemma-7b-it
q. Quit
```

Figura 2: Selección del modelo durante la configuración inicial con Groq.

```

Enter the number of your choice or 'q' to quit: 2
Enter your GROQ API key (press Enter to skip): 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
API keys and model saved to .env file
Selected model: groq/llama-3.1-70b-versatile
- Created prueba\.gitignore
- Created prueba\pyproject.toml
- Created prueba\README.md
- Created prueba\knowledge\user_preference.txt
- Created prueba\src\prueba\__init__.py
- Created prueba\src\prueba\main.py
- Created prueba\src\prueba\crew.py
- Created prueba\src\prueba\tools\custom_tool.py
- Created prueba\src\prueba\tools\__init__.py
- Created prueba\src\prueba\config\agents.yaml
- Created prueba\src\prueba\config\tasks.yaml
Crew prueba created successfully!

```

Figura 3: Selección de la API KEY del modelo Groq.

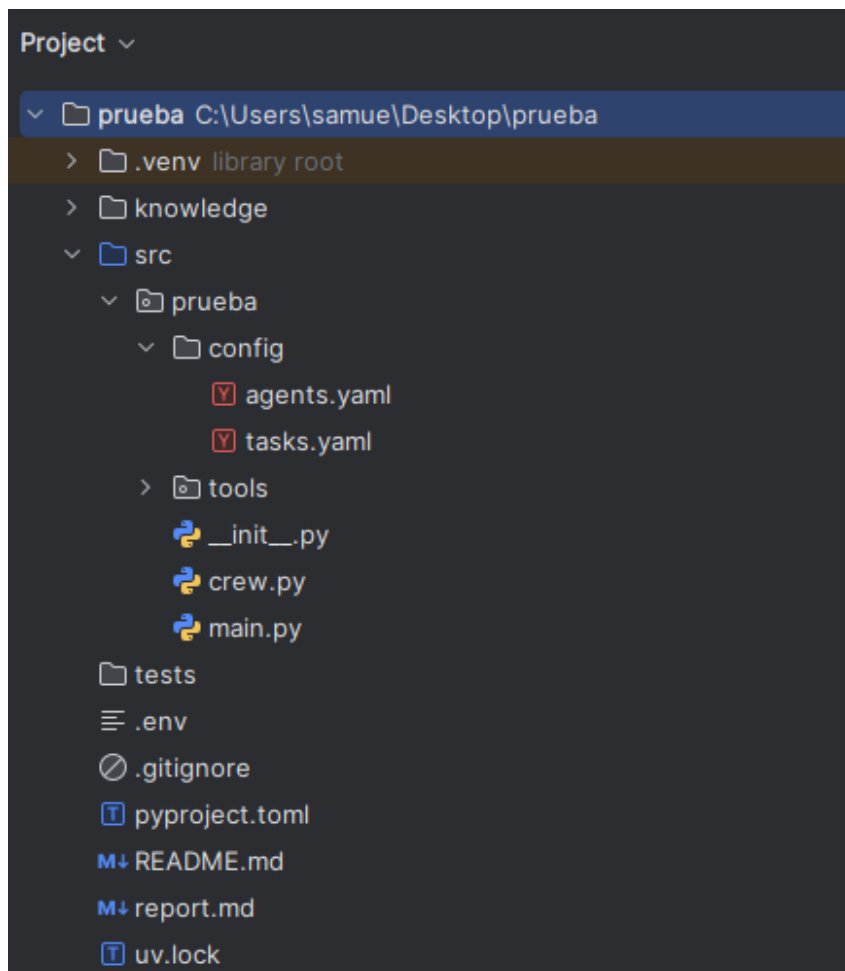


Figura 4: Estructura de proyecto generada automáticamente por CrewAI.

Con este comando se genera la base del proyecto (por ejemplo, `tfg`) y, para esta propuesta, los archivos clave son los siguientes:

- `.env`: almacena variables de entorno y claves de API.
- `src/tfg/main.py`: punto de entrada para ejecutar el flujo principal.

- **src/tfg/crew.py**: archivo de orquestación donde se define la *crew* y su ejecución.
- **src/tfg/config/agents.yaml**: definición de agentes (rol, objetivo y contexto).
- **src/tfg/config/tasks.yaml**: definición de tareas y asignación de cada tarea a los agentes.

6.3. Configuración de la prueba: dos agentes y dos tareas

Tras preparar el proyecto base, la prueba se definió con una configuración sencilla: **dos agentes** y **dos tareas** encadenadas de forma secuencial.

En esta configuración, el tema de trabajo se maneja mediante el parámetro `{topic}`. Para la ejecución concreta de esta prueba, en `main.py` se estableció el valor: *“Los misterios sin resolver del fondo del océano”*.

En `agents.yaml` se configuraron los siguientes perfiles:

- **researcher**: agente orientado a encontrar información llamativa, poco conocida e impactante sobre un tema concreto (`{topic}`).
- **reporting_analyst**: agente orientado a transformar los hallazgos en un guion dinámico para vídeo, con enfoque en retención de audiencia.

En `tasks.yaml` se definieron estas tareas:

- **research_task**: investigar `{topic}` y devolver **5 datos sorprendentes**, cada uno con título atractivo y explicación.
- **reporting_task**: utilizar esos 5 datos para generar un **guion completo de YouTube** (aprox. 5 minutos), con estructura de intro, desarrollo y outro, e indicaciones audiovisuales entre corchetes.

La orquestación en `crew.py` vincula cada tarea con su agente correspondiente y ejecuta el flujo con `Process.sequential`. En esta configuración, primero se ejecuta la investigación y, a continuación, se genera el guion final. Además, el resultado de la segunda tarea se guarda en `report.md`, lo que facilita la revisión posterior del experimento.

Tras generar el proyecto, se instalan dependencias internas desde la raíz:

```
crewai install
```

Finalmente, la ejecución de la primera prueba se lanza con:

```
crewai run
```

6.4. Resultado observado en la ejecución

En la ejecución mostrada en consola, el flujo se completó correctamente de principio a fin. Se observó el arranque de la *crew*, la ejecución de `research_task` por parte del agente `researcher` y, después, la ejecución de `reporting_task` por el agente `reporting_analyst`, manteniendo el orden secuencial definido en `crew.py`.

Como salida intermedia, el primer agente generó una lista de datos sorprendentes sobre el tema configurado. A continuación, el segundo agente utilizó esa información para construir el guion completo con formato tipo YouTube (intro, desarrollo y cierre), incluyendo indicaciones audiovisuales entre corchetes. El resultado final quedó reflejado tanto en la salida de consola como en el archivo de salida configurado (`report.md`).

También se observó que el trazado de ejecución (*tracing*) estaba desactivado por defecto, lo cual no impidió la ejecución del flujo, pero limita el nivel de inspección detallada del proceso.

Con este entorno preparado, ya se dispone de una base operativa para definir agentes, tareas y flujo de colaboración en la prueba práctica.

7. Conclusión